Anotações do Curso da NVIDIA - Agentic AI Explained

Maximilian Kenzo Abe Andrzejewski

O que exatamente é um Agent? *(What Is an Agent?)*

Definição: um agent é uma entidade de software dentro de um sistema de IA autônomo que aprende e atua em seu ambiente. Pode ser desde um programa simples até um **digital human** (humano digital) completo. Um agent executa um loop simples e **iterativo** (repete em ciclos).

O loop do agent (iterativo): Percebe → Raciocina → Atua → Aprende

- **Percebe:** recebe e interpreta o ambiente (prompt do usuário, dados de sensores em tempo real, informações de uma API externa). É como o agent "vê" o mundo.
- **Raciocina:** processa o que percebeu com base em regras internas ou lógica: faz computações, decompõe problemas complexos e avalia ações possíveis para decidir o próximo passo.
- **Atua:** altera o ambiente dentro de suas capacidades e limitações: usa uma ferramenta, gera uma resposta ou executa um comando.
- **Aprende:** melhora e se adapta ao longo do tempo via observação e feedback, usando **machine learning** (aprendizado de máquina), **deep learning** (aprendizado profundo) e **reinforcement learning** (aprendizado por reforço).

Ideia-chave: o design é flexível, então agents podem ser simples ou complexos. A classe que ficou muito popular recentemente combina software tradicional + entrada humana + **LLMs** (*large language models*, grandes modelos de linguagem).

> **Para fixar:** Percebe, Raciocina, Atua, Aprende (PRAA). Note que o "Aprende" é o passo que diferencia um agent de um simples script que só reage a entradas.

IA Agêntica vs. IA Generativa *(Agentic AI vs. Generative AI)*

Ideia central: Generative AI (cria conteúdo) + **Agentic AI** (decide e age) = **raciocínio semântico sob demanda**. A IA entende tarefas complexas de linguagem e visão e as traduz em etapas acionáveis, ligando a linguagem natural às interfaces rígidas de software.

Visão de sistema: não é só sobre agents individuais, mas sobre estruturar sistemas inteiros: software + modelos de IA + supervisão **human-in-the-loop** (humano no ciclo). Fica entre a engenharia de software e a gestão de projetos.

Contexto: agents existem na computação há décadas. O que mudou é combiná-los com Generative AI, o que os torna muito mais poderosos.

O que os LLMs permitem:

- **Raciocínio semântico sob demanda:** entende entradas complexas (texto ou imagem) e as divide em etapas acionáveis.
- **Ponte entre linguagem natural e software:** o agent traduz o pedido direto em ação, sem uma pessoa converter em código.
- **Trabalho em paralelo:** o problema é decomposto em várias tarefas resolvidas ao mesmo tempo por diferentes agents.
- **Observabilidade:** cada decisão pode ser rastreada, registrada e revisada, o que é essencial para confiança e conformidade.
- **Human-in-the-loop:** o agent pode pausar para supervisão humana onde julgamento ou responsabilidade pesam mais.

Em resumo: a combinação faz os agents saírem da **automação roteirizada** (scripts fixos) para a **colaboração adaptativa**.

> **Para fixar:** os 5 poderes dos LLMs em agents: Raciocinar, Traduzir, Paralelizar, Rastrear, Pausar (para humanos).

Principais Características de um Agent *(Core Traits of an Agent)*

Ideia central: os agents se distinguem por quatro características, que correspondem às etapas do loop (percebe, raciocina, atua, aprende).

- **Local Perception** (percepção local): o primeiro trabalho do agent é perceber o ambiente consumindo entradas (pergunta digitada, dados de sensor, informações de uma API externa). É como ele "vê" o mundo.
- **Internal Reasoning** (raciocínio interno): o "cérebro" do agent. Com base em um conjunto de regras, analisa o estado atual, divide problemas complexos em partes menores e decide o melhor curso de ação.
- **Local Action** (ação local): executa uma ação com base no raciocínio e assim interage com o ambiente e o altera. Exemplos: gerar uma resposta, alterar uma configuração, enviar um comando a outro sistema.
- **Learn and Adapt** (aprender e adaptar): quem projeta o agent pode combinar deep learning, reinforcement learning e **knowledge graphs** (grafos de conhecimento) para criar sistemas que se autodirecionam, mantêm consciência situacional e melhoram continuamente.

> **Para fixar:** Perception, Reasoning, Action, Learn/Adapt (percepção, raciocínio, ação, aprender/adaptar). É o mesmo ciclo do primeiro tópico, só que agora descrito como características do agent.

Arquitetura de IA Agêntica *(Agentic Architecture)*

Ideia central: a Agentic AI é modular, construída como blocos de Lego, o que a torna escalável e interoperável. Cada componente (camada) tem um propósito definido.

- **Camada de Percepção:** os "olhos e ouvidos". Coleta entradas de usuários, sensores ou APIs.
- **Camada de Decisão:** o "cérebro". Raciocina sobre as entradas, pesa alternativas e planeja os próximos passos.
- **Camada de Ação:** as "mãos". Executam: escrever texto, enviar uma chamada de API ou rodar código.
- **Camada de Aprendizado:** o ciclo de feedback. É um processo de autocorreção que compara resultados com metas e se ajusta.

> **Para fixar:** Olhos e ouvidos, Cérebro, Mãos, Feedback. São as mesmas quatro etapas do loop (percebe, raciocina, atua, aprende), agora vistas como camadas de arquitetura. Como são modulares, dá para trocar ou melhorar uma camada sem refazer o sistema inteiro.

Princípios de Engenharia *(Engineering Principles)*

Ideia central: para criar agents robustos, a engenharia usa quatro princípios.

- **Agentic Decomposition** (decomposição agêntica): dividir objetivos complexos em funções e tarefas especializadas, em vez de pedir a um único sistema para fazer tudo. Às vezes um único prompt bem feito resolve, mas nem sempre é a melhor opção. Quem desenvolve pode quebrar o problema em subtarefas lógicas e direcioná-las para as melhores ferramentas de cada uma. Isso usa os LLMs pelos seus pontos fortes e vai além de regras rígidas, rumo a uma computação adaptável, consciente do contexto e orientada a objetivos.
- **Context Engineering** (engenharia de contexto): escolher com cuidado o que o agent "vê": memória, instruções e dados relevantes, para mantê-lo focado e preciso. O agent pode receber contexto de fundo, recuperações (retrieval), instruções dinâmicas e configurações que influenciam as chamadas ao LLM, o estado que ele mantém e as rotinas de código que executa.
- **Workflow Design** (design de fluxo de trabalho): mapear as tarefas passo a passo, planejar falhas e tornar o processo observável, para rastrear cada decisão. O agent nem sempre funciona sem problemas, e pode ser preciso controle de qualidade para prestação de contas. Poder envolver pessoas e software quando necessário é requisito fundamental, e esse processo deve ser o mais contínuo possível.
- **Model Selection** (seleção de modelo): escolher a ferramenta certa para o trabalho. Os maiores modelos servem como padrão geral, com raciocínio avançado, mas boa parte do design agentic é escolher os módulos mais eficientes para a tarefa, ou criar um com resistência mínima (mais simples) que opere de forma previsível e confiável na sua função.

> **Para fixar:** Decomposição, Contexto, Workflow, Modelo. Em uma frase: quebre o problema, controle o que o agent vê, planeje o fluxo (inclusive as falhas) e use o modelo certo, não o maior por padrão.

Princípios de Engenharia em Detalhe *(Engineering Principles in Detail)*

Ideia central: aprofunda três dos quatro princípios: Context Engineering, Workflow Design e Model Selection.

1. Engenharia de Contexto *(Context Engineering)*

Definição: projetar com precisão o que o agent "vê" em cada etapa, dando a informação certa no momento certo, sem sobrecarregar com dados desnecessários. Analogia: a *mise en place* do chef.

Componentes do contexto (vai além do prompt do usuário):

- **Instruções do Sistema:** papel, objetivos e regras do agent.
- **Entrada do Usuário:** a solicitação atual.
- **Memória de Curto Prazo:** etapas e diálogos recentes da conversa atual (dá continuidade).
- **Memória de Longo Prazo:** fatos, preferências e resultados de interações passadas (dá personalização).
- **Conhecimento Recuperado:** documentos e dados de fontes externas (base da empresa, banco público).
- **Definições de Ferramentas e Outputs** (saídas): ferramentas disponíveis e resultados do uso mais recente.

Por que importa: o contexto pode ser um gargalo de desempenho, e o LLM depende muito da informação recebida.

- **Previne alucinações:** dados factuais reduzem o risco de o modelo inventar.
- **Mantém a continuidade:** evita que o agent "esqueça" o que já fez.
- **Garante alinhamento de objetivos:** papel e metas explícitos mantêm o foco.

2. Design de Fluxo de Trabalho *(Workflow Design)*

Definição: o plano de como o agent (ou sistema de agents) executa uma tarefa complexa: decompõe o objetivo em etapas menores e define o fluxo de informação entre elas. Um bom fluxo é modular, resiliente e observável.

Aspectos fundamentais:

- **Decomposição do Objetivo** (*goal decomposition*): do resultado desejado para sub-decisões e ações. Ex.: buscar artigos → resumir cada um → sintetizar → redigir o relatório.
- **Atribuição de Responsabilidades:** o que fica com o agent, com uma ferramenta externa ou com um humano (ex.: revisão de documento jurídico com human-in-the-loop).
- **Tratamento de Casos Excepcionais:** o que fazer se uma etapa falha, expira ou dá resultado ambíguo. Deve "falhar graciosamente" (*fail gracefully*): tentar de novo (retry) ou sinalizar para um humano.
- **Garantia de Observabilidade:** registrar cada prompt, resposta e chamada de ferramenta para rastrear decisões (depuração, qualidade, conformidade).

Por que importa:

- **Estrutura e confiabilidade:** um único prompt não dá conta de uma tarefa como "pesquisar e redigir um relatório de mercado".
- **Resiliência:** sistemas reais falham, e o fluxo antecipa isso para um passo falho não travar tudo.
- **Transparência:** fluxo claro permite rastrear decisões e gera confiança.

3. Seleção de Modelo *(Model Selection)*

Definição: escolher o modelo certo para a tarefa certa, usando o mais eficiente e econômico em cada passo. Funciona como a triagem em um hospital: sistema em níveis, em vez de um modelo grande e caro para tudo.

Flagship Models (modelos de ponta; os "grandes pensadores", ex.: GPT, Claude Opus, Gemini, Llama): poderosos para planejamento, síntese e resolução criativa de problemas. Suporte nativo a:

- **Reasoning** (raciocínio): fase de "pensamento" separada da fase de "resposta".
- **Tool-calling** (chamada de ferramentas): chama ferramentas automaticamente; é um caso especial de **structured output** (saída estruturada, restrita a uma gramática, como assinaturas de função).
- **Inference-time scaling** (escalonamento em tempo de inferência): acessa bancos externos, gera hipóteses e cadeias de pensamento por tentativa e erro, e roda trilhas em paralelo.
- **Multimodal support** (suporte multimodal): ingere imagens, áudio e documentos complexos (PDFs, apresentações).
- Por serem mais caros e com maior latência, usar com moderação: protótipos iniciais e etapas de alto valor.

Specialized Models (modelos especializados): modelos menores (**open-source**, de código aberto, ou **fine-tuned**, ajustados para uma tarefa) que podem ter desempenho igual ou melhor em casos específicos. São mais baratos, mais rápidos e mais fáceis de restringir e auto-hospedar. Bons para classificação, extração de dados, formatação e sumarização.

Roteamento por complexidade (*routing*): ex.: um modelo pequeno classifica a solicitação como "problema de suporte" e só chama um modelo de reasoning se precisar de solução complexa em várias etapas.

Por que importa:

- **Otimiza o desempenho:** tarefas simples rodam melhor em modelo pequeno (menos latência).
- **Reduz custos:** o LLM poderoso fica reservado para reasoning e planejamento.
- **Aumenta a escalabilidade:** modelos baratos cobrem as tarefas comuns e os poderosos ficam livres para o que fazem melhor.

> **Para fixar:**
>
> - **Contexto:** a informação certa, na hora certa (Sistema, Usuário, Curto prazo, Longo prazo, Recuperado, Ferramentas).
> - **Workflow:** plano modular, resiliente e observável (Decompor, Atribuir, Tratar falhas, Observar).
> - **Modelo:** o menor modelo que resolve a tarefa; flagship só para o que é complexo.

Padrões de Design Agênticos *(Agentic Design Patterns)*

Definição: modelos fundamentais para criar sistemas de IA em que agents autônomos ou semi-autônomos trabalham juntos. São padrões arquiteturais e comportamentais: orientam como os agents interagem com ferramentas, ambiente e outros agents. Promovem escalabilidade, modularidade e adaptabilidade ao formalizar workflows e interações comuns.

Ideia da equipe: um agent pode ser membro de uma equipe coordenada, com as ferramentas e processos necessários para problemas complexos. Os padrões dão a estrutura dessa equipe, seja um único agent numa tarefa ou um grupo gerenciando um workflow intrincado.

Os padrões recorrentes

1. Reflection Pattern (padrão de reflexão)

- O agent avalia criticamente e refina o próprio trabalho, num processo contínuo de autoaperfeiçoamento.
- Depois de concluir a tarefa, analisa a saída para achar o que refinar ou decidir se está suficiente e segura para ser a resposta "final".
- Resultado: respostas mais confiáveis ao longo do tempo.
- Em apps voltados ao usuário, é comum uma versão leve: um sistema de **guardrails** (barreiras de proteção) examina a saída do modelo e pode enfileirar uma fase de reescrita.

2. Tool Use Pattern (padrão de uso de ferramentas)

- O agent estende sua funcionalidade usando sistemas externos (bancos de dados, APIs, calculadoras), em vez de se limitar à programação interna.
- Permite fazer cálculos complexos, acessar dados atuais e executar ações no mundo real (enviar e-mail, fazer reserva).
- **Analogia:** a caixa de ferramentas do carpinteiro. Ele tem a habilidade, mas precisa das ferramentas para construir bem.
- Torna o agent mais adaptável e capaz de tarefas que seriam impossíveis sozinho.

3. Hierarchical Pattern (padrão hierárquico)

- Delega subtarefas a agents especializados em um sistema em camadas, dividindo metas grandes em partes gerenciáveis.
- Agents de nível superior cuidam da estratégia geral; agents de nível inferior executam as tarefas específicas.

- **Analogia:** estrutura corporativa. O CEO define metas, os chefes de departamento (Marketing, Finanças) dividem as tarefas com suas equipes, e cada membro entra com sua expertise.

4. Reactive Pattern (padrão reativo)

- O agent responde ao ambiente de forma instantânea e automática, sem depender de memória ou experiências passadas.
- Usa dados em tempo real dos sensores para ajustar o comportamento na hora. É essencial em situações dinâmicas e de mudança rápida.
- Exemplos: veículos autônomos, gerenciamento de tráfego, termostatos inteligentes, que respondem sem "pensar" no que aconteceu antes.

> **Para fixar:**
>
> - **Reflection:** o agent se autoavalia e refaz (revisa o próprio trabalho).
> - **Tool Use:** o agent usa ferramentas externas (caixa de ferramentas).
> - **Hierarchical:** chefes definem a estratégia, subordinados executam (organograma).
> - **Reactive:** reage ao agora, sem memória (sensor, ação imediata).

Um jeito de diferenciar: Reflection olha para trás (revisa), Reactive olha só para o presente, Tool Use olha para fora (recursos externos) e Hierarchical olha para cima e para baixo (níveis de comando).

Papéis Agênticos e Colaboração *(Agentic Roles and Collaboration)*

Ideia central: em sistemas **multi-agent** (multiagentes), **papéis** (*roles*) modulares e especializados trabalham juntos como membros de uma equipe. Um app agentic pode ter todos, vários ou nenhum desses papéis, dependendo dos objetivos. Os agents participantes usam **tools** (ferramentas), LLMs, APIs ou outros agents. Quando os papéis colaboram, formam **agentic pipelines** (fluxos agênticos), capazes de tarefas complexas de ponta a ponta com precisão e eficiência.

Os 5 papéis

1. Planner (planejador): o estrategista

- Começa tudo: faz a goal decomposition, quebrando uma meta grande em etapas gerenciáveis. **Analogia:** gerente de projeto.
- Ex.: "resumir o feedback recente dos clientes" → coletar avaliações de várias fontes → rodar **sentiment analysis** (análise de sentimento) → identificar temas recorrentes → redigir o relatório.
- Transforma objetivos vagos em workflows estruturados.
- É adaptativo: pode reordenar ou adicionar etapas conforme as condições mudam.
- Sem ele, o sistema corre o risco de perder tempo ou perder o foco.

2. Executor: quem faz

- Segue as instruções passo a passo: são os "trabalhadores braçais" do ecossistema agentic.
- Ex.: o Planner diz "analisar sentimento" → o Executor busca o dataset, aplica o modelo e devolve os resultados.
- Pode gerar texto, rodar código, chamar APIs ou coordenar subtarefas.
- Ponto forte: precisão. Entrega o que foi atribuído, sem divagar.
- Vários Executors podem rodar em paralelo, o que aumenta a eficiência e a escalabilidade.

3. Critic / Evaluator (crítico / avaliador): o editor

- O controle de qualidade embutido: revisa as saídas antes de finalizar, como um editor revisa um rascunho.
- Checa precisão, clareza, consistência e alinhamento com a intenção original.
- Ex.: confere fatos, estilo e formatação de um relatório de mercado. Se algo estiver errado, devolve para revisão.
- Gera confiança ao pegar erros cedo, antes que pequenas falhas virem problemas maiores.

4. Tool-User / Connector (usuário de ferramentas / conector): a ponte com o mundo externo

- Liga o agent a sistemas externos. Sozinho, um LLM só gera texto, mas com tools pode agir: consultar bancos de dados, puxar dados ao vivo, enviar e-mails, rodar código.
- Ex.: um agent de planejamento de viagem não só sugere "veja o clima", mas chama uma API de clima e atualiza o roteiro.
- Transforma o agent de um "mecanismo de sugestão" em um colaborador com poder de ação.

5. Coordinator (coordenador): o maestro

- Gerencia como os agents interagem: faz o routing das tarefas, resolve conflitos e mantém os workflows alinhados.
- Sem ele, os agents podem duplicar trabalho ou puxar em direções diferentes.
- Sincroniza tudo: passa o **state** (estado), junta os outputs e entrega o resultado final.
- Essencial em sistemas multi-agent com dezenas de agents em paralelo: garante eficiência, alinhamento e confiabilidade.

Exemplo completo (análise de mercado): o Planner desenha as etapas → o Executor analisa os dados → o Critic revisa o rascunho → o Tool-User busca evidências de apoio → o Coordinator sincroniza tudo e entrega o resultado.

> **Para fixar:** Planner (planeja), Executor (faz), Critic (revisa), Tool-User (conecta), Coordinator (orquestra).

Um jeito de lembrar o fluxo: planejar → executar → revisar, com o Tool-User dando acesso ao mundo externo e o Coordinator regendo todo mundo por cima.

Aplicações no Mundo Real *(Real-World Applications)*

Ideia central: a Agentic AI não é só teoria. Ela já está em uso em vários setores.

- **Telecommunications** (telecomunicações): agents configuram e otimizam redes de forma autônoma, reduzindo a complexidade e o risco de erro humano.
- **Enterprise Data Flywheels** (ciclos de dados corporativos): agents curam, analisam e devolvem os dados aos processos da empresa, criando ciclos de melhoria contínua.
- **Customer Experience** (experiência do cliente): assistentes de IA agilizam workflows como agendamento, suporte e onboarding, liberando as pessoas para trabalhos de maior valor.
- **Governance and Compliance** (governança e conformidade): plataformas de **traceability** (rastreabilidade) garantem que cada decisão do agent possa ser monitorada, auditada e explicada, o que é essencial em setores regulados.

> **Para fixar:** Telecom (redes), Data Flywheel (dados em ciclo), Customer Experience (atendimento), Governance (rastreio e auditoria).

O Data Flywheel é o passo "Aprende" do loop do agent em escala de empresa, e a Governance depende da observabilidade do Workflow Design, já que sem registrar cada decisão não há como auditar.

Construindo e Escalando *(Building and Scaling)*

Ideia central: a jornada começa simples. O ponto de partida é um **agent loop** básico: um único agent que recebe uma entrada em linguagem natural e segue um ciclo.

O agent loop básico

1. **Registra a entrada no history** (histórico). O histórico começa com informações do sistema: o papel do agent, sua **directive** (diretriz) e as ações que ele pode tomar.
2. **Envia o histórico ao LLM.** Ele é condensado em um prompt, e o agent espera o resultado.
3. **Interpreta a resposta do LLM** (*parsing*) e classifica qual ação deve tomar.
4. **Se escolher uma ação,** executa e adiciona a ação e o resultado ao histórico.
5. **Se nenhuma ação for escolhida** e sair uma resposta direta, devolve o resultado como output e também o adiciona ao histórico. Na próxima chamada, o agent lembra do histórico anterior.

ReAct (Reason-and-Act)

Esse ciclo é o que se chama de **ReAct loop** (*Reason and Act*, "raciocinar e agir"). A partir dessa base, é possível adicionar camadas de memória, planejamento e integração de tools para criar agents mais sofisticados.

Como escalar

- O pipeline pode se ramificar em uma cascata de workflows e uma hierarquia de abstrações.
- Ou pode ser simplificado a uma única chamada de tool, sem loops e sem histórico.
- **Comece pequeno:** construa um único papel e depois expanda.
- Outra opção: partir de um sistema já existente e identificar onde um passo manual é só uma classificação ou uma reformulação de uma entrada em linguagem natural (esses pontos são bons candidatos para um agent).

> **Para fixar:**
>
> - **Loop básico:** registrar → mandar ao LLM → classificar a resposta → agir (ou responder) → guardar no histórico.
> - **ReAct** = Reason + Act, o loop que serve de base para agents mais complexos.
> - **Regra de ouro:** comece pequeno, com um papel, e vá adicionando memória, planejamento e tools.
> - Esse loop é a versão prática do Percebe → Raciocina → Atua, e o histórico faz o papel de memória de curto prazo da Engenharia de Contexto (Context Engineering).

Frameworks e Ferramentas *(Frameworks and Tools)*

Ideia central: não é preciso reinventar a roda. Existem **frameworks** que já entregam a infraestrutura para construir agents, para você focar no design do comportamento do agent e não na infraestrutura.

O que eles oferecem: infraestrutura para routing, memória e **orchestration** (orquestração).

Os principais:

- LangChain
- LangGraph
- Crew.ai
- **NVIDIA NeMo Agent Toolkit:** estende essa base com componentes prontos para uso corporativo (*enterprise-ready*).

> **Para fixar:** frameworks cuidam de routing, memória e orquestração, e você cuida do comportamento do agent. Nomes para guardar: LangChain, LangGraph, Crew.ai e NeMo Agent Toolkit (o da NVIDIA, voltado para empresas).

Tendências Atuais e Conclusão *(Current Trends and Conclusion)*

Tendências atuais em Agentic AI

O campo avança rápido. Algumas tendências emergentes:

- **Persistent-Memory Agents** (agents com memória persistente): sistemas que retêm conhecimento entre sessões e se adaptam ao longo de períodos longos.
- **Multimodal Agents** (agents multimodais): combinam texto, fala, visão e outras modalidades para uma interação mais rica.
- **Human-in-the-Loop Systems:** misturam automação com supervisão em tarefas de alto risco, como saúde e finanças.
- **Agent Communication Protocols** (protocolos de comunicação entre agents): novos padrões para colaboração entre agents, permitindo grandes redes de agents trabalhando juntos em ecossistemas dinâmicos.

Desafios éticos

Com os avanços vêm desafios éticos. **Alignment** (alinhamento), responsabilização e transparência são essenciais para que os sistemas agentic continuem seguros e confiáveis. **Audit logs** (registros de auditoria), interpretabilidade e supervisão humana não são opcionais: fazem parte da base de uma Agentic AI responsável.

Conclusão

- A Agentic AI é um salto na forma de pensar a inteligência artificial. Em vez de ferramentas que fazem previsões ou geram conteúdo, os agents são participantes de um sistema maior, que interage diretamente com software, hardware e pessoas.
- Seja otimizando uma rede de telecomunicações, apoiando decisões corporativas ou atendendo clientes, esses sistemas já estão transformando setores.
- Conforme evoluem (memória, multimodalidade, colaboração), trazem novas possibilidades e novas responsabilidades.
- **Em resumo:** Generative AI cria conteúdo. Agentic AI faz as coisas acontecerem.

Próximos passos *(Call to Action)*

- Explorar os notebooks do NVIDIA Deep Learning Institute (DLI).
- Experimentar construir um agent simples: começar com um loop, adicionar memória, adicionar tools e deixar a curiosidade guiar.
- Quando estiver pronto, compartilhar o trabalho e colaborar no NVIDIA AI developer Forum.

Cursos self-paced (no seu ritmo):

- Building RAG Agents with LLMs
- Evaluating RAG and Semantic Search Systems
- Build a Deep Research Agent

Workshops com instrutor:

- Building Agentic AI Applications with LLMs
- Building AI Agents with Multimodal Models

Fórum: Agentic AI Developer Forum

> **Para fixar:**
>
> - **Tendências:** memória persistente, multimodal, human-in-the-loop, protocolos de comunicação entre agents.
> - **Ética:** alignment, responsabilização, transparência (com audit logs, interpretabilidade e supervisão humana como base).
> - **Frase-chave do curso:** Generative AI cria conteúdo; Agentic AI faz as coisas acontecerem.

LINK DO CURSO:

https://learn.nvidia.com/courses/course?course_id=course-v1:DLI+S-FX39+V1&unit=block-v1:DLI+S-FX39+V1+type@vertical+block@d0648d8eb8fc4aac9be870c0436aef1c

Comprovante de conclusão do curso (10/10/2026):

